

ENTERPRISE SYSTEMS

(Spring 2025)

Solution by (Abeerah Fatima)

Q.1. Answer the following short questions.

(6x5=30)

1) What is Data Reader?

A Data Reader is a read-only, forward-only object used to retrieve data from a database very quickly. It reads data one row at a time directly from the database without storing the entire result in memory.

Key Features:

- It is read-only — you cannot update data using a Data Reader.
- It is forward-only — you can only move forward through the rows, not go back.
- It is fast and uses less memory compared to storing all data at once.
- It requires an open database connection while reading data.

Example: In .NET (C#), SqlDataReader is used to fetch rows from a SQL Server database one by one. It is ideal for displaying large amounts of data quickly.

2) What is Entity Data Model?

Entity Data Model (EDM) is a framework used to describe the structure of data in a system. It represents data as Entities (objects/things) and the Relationships between them.

Key Concepts:

- Entity: A real-world object, e.g., Student, Product, Order.
- Attribute: Properties of an entity, e.g., Student has Name, Age, ID.
- Relationship: How two entities are connected, e.g., a Student enrolls in a Course.

In Enterprise Systems (like Microsoft Entity Framework), EDM is used to map database tables to programming objects automatically. This makes it easy for developers to work with data using code instead of writing SQL queries manually.

3) What is a Delegate in Enterprise Systems?

A Delegate is a type that holds a reference to a method (function). It works like a pointer to a function — you can use it to call a method indirectly. Delegates are heavily used in event-driven programming and enterprise systems.

Simple Explanation: Think of a delegate as a variable that stores a method name. When you call the delegate, it automatically calls that method.

Uses in Enterprise Systems:

- Event Handling: When a button is clicked, a delegate is used to call the correct function.
- Callback Functions: Used to pass methods as parameters.
- Decoupling: Helps separate different parts of a system so they can work independently.

Example: In C#, a delegate is declared as: `delegate void MyDelegate(string msg);` — this can then call any method that matches this signature.

4) Write the Difference between WSDL and SOAP.

Feature	WSDL	SOAP
Full Form	Web Services Description Language	Simple Object Access Protocol

Purpose	Describes what a web service does (like a manual)	Used to send/receive data between systems (like a vehicle)
Type	Describes the service interface	A messaging protocol
Format	Written in XML to describe operations	Uses XML to carry messages
Role	Tells the client how to use the service	Carries the actual request and response
Analogy	Menu of a restaurant (tells what is available)	Waiter who carries orders to kitchen and back

5) What is the Syntax of XML?

XML (eXtensible Markup Language) is used to store and transport data. Its syntax rules are:

- Every XML document must have one Root element that wraps all other elements.
- Every opening tag must have a closing tag. Example: <name>Ali</name>
- Tags are case-sensitive. <Name> and <name> are different.
- All elements must be properly nested (no overlapping tags).
- Attribute values must be in quotes. Example: <student id="101">

Basic XML Structure Example:

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student id="101">
    <name>Ali</name>
    <age>20</age>
  </student>
</students>
```

6) What is Web API?

A Web API (Application Programming Interface) is a set of rules and tools that allows two applications to communicate with each other over the internet using HTTP (the same protocol used by websites).

Simple Explanation: Think of a Web API like a waiter in a restaurant. You (client) order food (send a request), the waiter (API) takes it to the kitchen (server), and brings back the food (response).

Key Points:

- Uses HTTP methods: GET (read data), POST (send data), PUT (update), DELETE (remove).
- Data is usually sent and received in JSON or XML format.
- Used to connect mobile apps, websites, and enterprise software.
- Example: A weather app on your phone uses a Web API to get weather data from a server.

In Enterprise Systems, Web APIs are used to connect different modules like inventory, billing, and customer management so they can share data with each other.

Q.2. Answer the following questions.

(2x15=30)

1. What is MVC? Write its Uses and Advantages.

What is MVC?

MVC stands for Model-View-Controller. It is a software design pattern used to organize the code of a web or desktop application. It divides the application into three separate components, each with a specific job.

The Three Components of MVC:

Part	Job	Example
Model	Handles data and business logic. Talks to the database.	Student class with Name, Age, GPA

View	What the user sees — the user interface (UI).	HTML page showing a list of students
Controller	Middle man. Receives user requests, calls Model, sends result to View.	StudentController.cs handling requests

Simple Real-Life Example: On a shopping website — when you search for a product (Controller receives the request), the database finds the products (Model), and the results are displayed on screen (View).

Uses of MVC

- Web Application Development: Frameworks like ASP.NET MVC, Laravel (PHP), and Ruby on Rails use MVC pattern.
- Enterprise Systems: Large systems use MVC to separate logic from presentation.
- Mobile Applications: Android and iOS apps use MVC to manage data and UI separately.
- E-commerce Platforms: Product listing, cart management, and order processing use MVC.
- Banking Systems: Handles complex business rules (Model) separately from the user interface (View).

Advantages of MVC

- Separation of Concerns: Each part (Model, View, Controller) does its own job. This makes code cleaner and easier to understand.
- Easy to Maintain: If you change the database, you only update the Model. The View and Controller stay the same.
- Easy to Test: Each component can be tested independently. You can test the Model without needing the UI.
- Multiple Views, One Model: You can create different interfaces (web, mobile, desktop) using the same Model.
- Team Work is Easier: Frontend developers work on View, backend developers work on Model — no conflicts.
- Code Reusability: The same Model or Controller can be reused in different parts of the application.
- Faster Development: Because responsibilities are clearly divided, development becomes faster.

2. How we define Variables in JavaScript? Write a Program to Calculate the Factorial of a Number.

Variables in JavaScript

A variable is a container that stores data (a value). In JavaScript, we can define variables using three keywords: var, let, and const.

Keyword	Usage	Can be changed?	Scope
var	Old way to declare variables	Yes (can be redeclared too)	Function scope
let	Modern, preferred for changeable values	Yes (value can change)	Block scope
const	For values that never change	No (value is fixed)	Block scope

Examples of Variable Declaration:

```
var studentName = "Ali";    // old style
let age = 20;               // modern, changeable
const PI = 3.14;           // constant, never changes
```

```
// Variables can store different data types:
let number = 10;           // Number
let name = "Sara";         // String
let isActive = true;       // Boolean
let marks = [85, 90, 78];  // Array
```

JavaScript Program to Calculate Factorial

What is Factorial? Factorial of a number n means multiplying all numbers from 1 to n .

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Method 1: Using a Loop (Simple)

```
// Factorial using a for loop
function calculateFactorial(n) {
    let result = 1;
    for (let i = 1; i <= n; i++)
        { result = result * i;
    }
    return result;
}

// Get number from user
let num = parseInt(prompt("Enter a number:"));

// Check for negative numbers
if (num < 0) {
    console.log("Factorial not possible for negative numbers.");
} else {
    console.log("Factorial of " + num + " = " + calculateFactorial(num));
}
```

Method 2: Using Recursion (Advanced)

```
// Factorial using recursion
function factorial(n) {
    if (n === 0 || n === 1)
        { return 1; // Base case
    }
    return n * factorial(n - 1); // Recursive call
}

let number = parseInt(prompt("Enter a number:"));
console.log("Factorial = " + factorial(number));
```

Output Example:

Input: 5

Output: Factorial of 5 = 120

Explanation: The function multiplies each number from 1 to 5 step by step: $1 \times 2 = 2$, then $2 \times 3 = 6$, then $6 \times 4 = 24$, then $24 \times 5 = 120$. The final answer is 120.